

이태호 포트폴리오

Phone: 010-4242-7531 · Email: dlxogh561@gmail.com · Portfolio: <https://havefunatcode.github.io/>

워크스페이스 Audit Log 시스템 설계 및 개발

Springboot Java Kotlin AWS SQS AOP OAS MSA

프로젝트 요약

- 개발 기간: 2026.03 ~ 2026.04
- 개발 내용:
 - 멀티테넌트·멀티모듈 환경에서 사용자 활동 이력을 통합 추적하는 신규 Bounded Context 설계·개발 전담
 - 6개 도메인 모듈 횡단 비동기 감사 이벤트 발행 파이프라인 구축
 - AOP 기반 도메인-감사 결합 분리
 - 트랜잭셔널 아웃박스 기반 발행 + SQS DLQ·수동 redrive 적재 실패 격리·복구 운영 체계
- 핵심 기술: AOP, Spring Event, Transactional Outbox, Amazon SQS, DLQ, OpenAPI Spec(Contract-First)
- 역할: 아키텍처 설계·개발 전담, 인프라 구성, 6개 도메인 모듈 적용 가이드 및 코드 리뷰

프로젝트 배경

엔터프라이즈 고객사의 보안 감사(GDPR / 국내 개인정보보호법 / 사내 컴플라이언스) 요구사항을 충족하기 위해, 12개 마이크로서비스로 구성된 차세대 SaaS에서 발생하는 모든 사용자 활동 이력을 통합 추적할 수 있는 Audit Log 체계가 필요했습니다.

기존 시스템은 도메인 모듈별로 로그 적재 방식이 제각각이었고, 감사 코드가 비즈니스 로직에 직접 박혀있어 도메인 코드의 가독성을 해치고 변경에 취약했습니다. 또한 일부 모듈에서는 이벤트 누락이 빈번하여 컴플라이언스 요건상 요구되는 추적 완전성을 충족할 수 없었고, 이로 인해 엔터프라이즈 타겟 고객사 3곳과의 신규 계약이 보안 요건 미달로 보류된 상황이었습니다.

이를 해결하기 위해 도메인 코드를 건드리지 않고 감사 이벤트를 안정적으로 적재할 수 있는 신규 Bounded Context를 설계했습니다.

프로젝트 설명

도메인 코드와 감사 코드의 결합을 어떻게 끊을 것인가?

처음에는 도메인 서비스 안에서 `AuditService.log(...)` 를 직접 호출하는 방식을 검토했지만, 도메인 변경이 발생할 때마다 감사 코드를 함께 수정해야 하고, 6개 모듈 × 수십 개 액션에 일관성 있게 적용하기가 현실적으로 어려웠습니다.

이에 **AOP 어노테이션**을 도입하여, 감사 대상 메서드에 어노테이션만 부착하면 별도 코드 없이 메서드 시그니처·실행 결과·실행 컨텍스트(테넌트, 사용자, IP, User-Agent)가 자동으로 수집되어 감사 이벤트로 발행되도록 설계했습니다.

이 구조의 장점은 다음과 같습니다:

- 도메인 코드와 감사 코드의 **관심사 완전 분리** → 도메인 PR 리뷰가 감사 도메인 지식 없이도 가능
- 신규 액션 추가 시 **어노테이션 한 줄로 적용** → 변경 비용 최소화
- 감사 정책 변경(예: 마스킹 필드 추가)이 AOP 한 곳에서 끝남

이벤트 유실을 어떻게 막을 것인가?

감사 로그는 "있어도 그만, 없어도 그만"인 데이터가 아니라 **컴플라이언스 증빙 자료**이기 때문에 누락에 극도로 민감한 요건이었습니다.

처음에는 동기적으로 DB에 직접 적재하는 방식을 고려했지만, 다음과 같은 단점이 있었습니다:

- 감사 적재 실패가 도메인 트랜잭션 실패로 이어져 사용자 경험에 영향
- DB 부하가 도메인 트랜잭션 처리량을 잠식

이에 트랜잭셔널 아웃박스 + Amazon SQS 비동기 발행 + DLQ + 수동 redrive 파이프라인으로 설계했습니다:

- AOP가 수집한 감사 이벤트를 도메인 트랜잭션 안에서 outbox 테이블에 함께 커밋 — 도메인 변경과 이벤트 기록이 원자적으로 묶여, 커밋과 발행 사이의 유실 구간을 구조적으로 제거
- 별도 발행기가 outbox를 읽어 SQS로 발행하고 발행 완료를 마킹 — 재발행 가능성은 컨슈머의 멍등 처리로 흡수
- 적재 컨슈머 장애 시 DLQ로 격리, 운영자가 원인 분석 후 수동 redrive
- 적재 실패 케이스를 시뮬레이션하여 격리·복구 경로 검증 (시뮬레이션 범위 내 누락 0건)

OAS Contract-First → DB-Driven 인프라 재구조화

신규 Audit Action을 추가할 때마다 공통 모듈에 Enum/DTO를 추가해야 했고, 이로 인해 6개 도메인 모듈이 모두 공통 모듈에 강하게 의존하는 구조가 되었습니다. 액션 1개 추가에 공통 모듈 → 전 도메인 모듈 빌드/배포가 필요한 상황이 발생할 것임을 사전에 발견했습니다.

이에 OAS Contract를 DB로 이관하여, Action 메타데이터를 코드가 아닌 데이터로 관리하도록 인프라를 재구조화했습니다. 이를 통해 신규 Action 추가가 도메인 팀 단독 PR로 완결되도록 단순화했습니다.

성과

- 보안 요건 미달로 보류되었던 엔터프라이즈 타겟 고객사 3곳과 신규 계약 체결에 기여
- 6개 도메인 모듈에 적용 완료, 도메인 코드 변경 없이 감사 이력 자동 수집
- 트랜잭셔널 아웃박스 발행 + DLQ·수동 redrive 체계 확립, 장애 시뮬레이션 범위 내 이벤트 누락 0건 확인
- 신규 Action 추가 리드타임: 공통 모듈 PR + 도메인 PR(다중 빌드) → 도메인 단독 PR(단일 빌드) 로 단순화

B2B SaaS Billing 시스템 설계 및 개발

Springboot Java Kotlin JPA PortOne SQS PessimisticLock Idempotency

프로젝트 요약

- 개발 기간: 2025.12 ~ 2026.02
- 개발 내용:
 - 차세대 AI 특허 분석 SaaS "키워드 인사이트" Billing 도메인 전체 설계
 - PortOne PG 연동 결제 엔진 (비관적 락 + 웹훅 멍등성)
 - 체크아웃-실행 분리 + @Lazy self 4단계 독립 트랜잭션
 - Strategy 패턴 기반 환불 시스템
 - SQS 기반 비동기 토큰 소모 아키텍처 (만료 임박 순 차감 + 멍등 컨슈머)
- 핵심 기술: PESSIMISTIC_WRITE, @Transactional(REQUIRES_NEW), AFTER_ROLLBACK, Amazon SQS, Strategy Pattern
- 역할: Billing 도메인 설계 및 개발 전담

프로젝트 배경

차세대 AI 특허 분석 SaaS의 수익 모델 근간이 되는 Billing 시스템 전체 설계가 필요했습니다. 시트 기반 구독 과금, PG 결제 연동, 환불 정책, AI 기능 사용에 따른 토큰 소모까지 복잡한 결제 도메인을 아키텍처 수준에서 설계해야 했고, 결제 도메인 특성상 단 1건의 정합성 깨짐도 직접적인 금전 손실로 이어지기 때문에 동시성·멍등성·실패 복원성을 최우선으로 고려했습니다.

프로젝트 설명

동시 결제 요청에서 정합성을 어떻게 보장할 것인가?

같은 워크스페이스에서 결제·시트 추가·플랜 변경 요청이 짧은 시간에 동시에 발생할 수 있는데, 이때 마지막 커밋이 이전 커밋을 덮어쓰는 lost update 문제가 발생할 수 있습니다.

낙관적 락(Optimistic Lock)을 검토했으나, 결제 도메인은 충돌 시 "재시도하세요"로 끝낼 수 없는 영역이라 판단했습니다. 충돌이 발생하면 PG 결제는 이미 성공한 상태에서 우리 시스템만 롤백되는 정합성 깨짐이 발생할 수 있기 때문입니다.

이에 **비관적 락(PESSIMISTIC_WRITE)**으로 결제 진입점부터 직렬화하여, PG 호출 이전에 워크스페이스 단위 락을 선점하도록 설계했습니다. 처리량은 줄지만, 결제 도메인에서는 **정합성 > 처리량**이라는 우선순위가 명확했습니다.

웹훅 멍등성을 어떻게 검증할 것인가?

PG 웹훅은 재시도 정책상 동일 결제 건에 대해 여러 번 호출될 수 있고, 네트워크 이슈로 응답이 누락된 경우에도 중복 호출이 발생합니다.

상태 기반 멍등성 검증을 도입하여, 결제 상태머신(PENDING → PAID → SETTLED)을 정의하고 **현재 상태에서 허용되지 않는 전이는 무시**하도록 처리했습니다. 별도 멍등성 키 테이블을 두지 않고도 상태 자체가 멍등성 보증 장치가 되도록 설계했습니다.

결제 실패 시 체크아웃 데이터를 어떻게 보존할 것인가?

초기 구현은 단일 트랜잭션 안에서 체크아웃 → PG 호출 → 시트 부여를 처리했는데, PG 호출 실패 시 메인 트랜잭션이 롤백되면서 **사용자가 입력한 결제 의도(체크아웃) 데이터까지 함께 사라지는** 문제가 있었습니다.

이를 해결하기 위해 **체크아웃-실행 분리 패턴**을 도입하고, @Lazy self 주입으로 한 클래스 안에서 **4단계 독립 트랜잭션**이 가능하도록 구조화했습니다:

1. 체크아웃 생성 (독립 커밋)
2. PG 호출 (외부 I/O)
3. 결제 결과 반영 (독립 커밋)
4. 시트 부여 (독립 커밋)

다만 @Lazy self 주입은 프로시 경우를 위한 차선택이라고 판단하고 있습니다. 트랜잭션 경계를 별도 빈으로 분리하는 것이 정석이며, 이를 개선 1순위로 두고 있습니다.

결제 실패 이력을 어떻게 유실 없이 남길 것인가?

결제 실패 케이스를 단순 로그로만 남기면 메인 트랜잭션 롤백 시 함께 사라져 사후 분석이 불가능했습니다. 운영 중 "왜 실패했는지 모르는" 실패가 가장 위험합니다.

이에 **@TransactionalEventListener(AFTER_ROLLBACK) + @Transactional(REQUIRES_NEW)** 조합으로, 메인 트랜잭션이 롤백되어도 실패 이력은 **새 트랜잭션에서 별도 커밋**되도록 처리하여, 메인 트랜잭션 롤백과 무관하게 실패 이력이 남는 구조를 만들었습니다.

환불 정책의 다양성을 어떻게 관리할 것인가?

플랜별·국가별·정책별 환불 규칙이 모두 달라 if/else로 처리하면 정책 변경마다 결제 도메인 코드 전체를 수정해야 했습니다.

Strategy 패턴을 도입하여 환불 규칙을 정책 객체로 분리하고, 신규 정책 추가가 **새 Strategy 구현체 추가**만으로 끝나도록 설계했습니다. 환불 정책은 비즈니스 요구로 자주 바뀌는 영역이라 변경 비용을 낮추는 것이 중요했습니다.

비동기 토큰 차감 시 동시성을 어떻게 처리할 것인가?

AI 기능 사용 시 토큰이 차감되는데, 동일 사용자가 동시에 여러 AI 요청을 보낼 경우 토큰 잔액 계산이 어긋날 수 있습니다. 또한 SQS Standard 큐는 At-Least-Once 보장이라 중복 메시지가 전제 조건입니다.

이에 **SQS 기반 비동기 토큰 소모 + 멍등 컨슈머** 구조로 설계했습니다:

- 만료 임박 토큰부터 우선 차감하는 순차 차감 알고리즘
- 비즈니스 키 기반 멍등 처리로 중복 메시지가 와도 차감이 두 번 일어나지 않도록 방지

운영 중 발생한 데드락 장애를 어떻게 해결했는가

운영 도중 토큰 동시 차감 시 데드락이 간헐적으로 발생했습니다. 분석 결과, **FK 제약의 공유 락(S-lock)**과 **UPDATE의 배타 락(X-lock)**이 서로 다른 순서로 획득되며 데드락이 발생하는 패턴이었습니다.

해결의 핵심은 락을 더 거는 것이 아니라 **락의 통제권을 확보**하는 것이었습니다. 획득 순서를 통제할 수 없는 암묵 락(FK의 S-lock)을 물리 FK 제거로 없애고(논리적 무결성은 애플리케이션 레벨 검증으로 방어), **SELECT ... FOR UPDATE**로 X-lock을 선점하여 락 획득 순서를 일관되게 만들었습니다. 이후 동일 패턴의 데드락은 재발하지 않았습니다.

성과

- 차세대 AI 특허 분석 SaaS의 **Billing 도메인 전체 설계·개발 완료**
- 운영 기간 중 대사(reconciliation) 배치 기준, 결제·시트·토큰 도메인의 정합성 불일치로 보고된 장애 **0건**
- 메인 트랜잭션 블록과 무관하게 실패 이력을 별도 트랜잭션으로 커밋하는 보존 구조 확립으로 운영 분석 가능성 확보
- 데드락 장애 원인 분석 및 근본 해결 — 이후 동일 패턴 재발 **0건**

차세대 특허 검색 서비스 백엔드 아키텍처 설계

Springboot

Kotlin

MSA

Kubernetes

ArgoCD

GitHubActions

SQS

Redis

CircuitBreaker

프로젝트 요약

- 개발 기간: 2025.08 ~ 2025.12
- 개발 내용:
 - 12개 마이크로서비스 기반 차세대 SaaS 백엔드 아키텍처 설계 참여 및 담당 도메인 구현
 - 동기(REST) / 비동기(SQS) 통신 정책 수립
 - Kubernetes + ArgoCD GitOps CD 파이프라인 전환 협업
 - GitHub Actions CI + Docker 이미지 빌드/배포 자동화
 - Redis 세션/인증 토큰 관리, Circuit Breaker 외부 호출 격리
- 핵심 기술: MSA, Kubernetes, ArgoCD, GitHub Actions, Amazon SQS, Redis, Resilience4j
- 역할: 백엔드 아키텍처 설계 참여, 담당 도메인 설계·구현

프로젝트 배경

기존 모놀리식 서비스는 단일 배포·단일 장애점 구조로 인해 확장성 한계가 명확했습니다. 검색 트래픽이 급증하는 시간대에 결제·번역·알림 등 무관한 도메인까지 영향을 받았고, 도메인별 독립 배포가 불가능해 릴리즈 사이클이 길었습니다.

신규 SaaS 제품(키워드 인사이트)의 요건과 **초거대 SaaS 정부과제** 심사를 모두 충족하기 위해, 차세대 서비스의 백엔드 아키텍처를 처음부터 재설계해야 했습니다.

프로젝트 설명

MSA 분리 기준을 어떻게 잡을 것인가?

"마이크로서비스로 쪼개라"는 답이 아니라, **어디서 쪼개야 하는가**가 핵심 질문이었습니다. 너무 잘게 쪼개면 분산 트랜잭션·네트워크 호출 비용이 폭증하고, 너무 크게 쪼개면 모놀리식의 단점이 그대로 남습니다.

DDD의 Bounded Context를 기준으로 12개 마이크로서비스(회원 / 결제 / 검색 / 번역 / 파일 / 알림 등)를 분리하고, 각 서비스가 **자체 데이터 저장소를 소유**하도록 설계했습니다. 도메인 경계가 곧 서비스 경계가 되도록 매핑하여, 변경의 진원지가 한 서비스 안에 머무르도록 했습니다.

서비스 간 통신을 동기/비동기 중 무엇으로 할 것인가?

동기 REST만 사용하면 호출 체인이 길어져 한 서비스 장애가 전체로 전파됩니다. 비동기 메시징만 사용하면 단순 조회까지 비동기로 처리되어 복잡도가 폭증합니다.

이에 **요청-응답 의미가 명확한 동기 호출은 REST API, 이벤트성·후속 작업은 Amazon SQS**로 명확히 역할을 분리했습니다. 또한 외부 시스템 호출(PG, 번역 API 등)에는 **Resilience4j Circuit Breaker**를 적용하여 외부 장애가 우리 시스템으로 전파되지 않도록 격리했습니다.

Docker Stack에서 Kubernetes로 왜 전환했는가?

기존 Docker Stack은 단순 컨테이너 오케스트레이션은 가능했지만, **롤링 배포 / 헬스체크 기반 트래픽 라우팅 / HPA 오토스케일링** 등 운영에 필수적인 기능이 부족했습니다. 12개 서비스를 동시에 운영하면서 수동 배포·수동 스케일링은 운영 부담이 너무 컸습니다.

이에 팀 차원에서 **Kubernetes 기반 컨테이너 오케스트레이션**으로 전환을 진행했고, 저는 전환 아키텍처 설계에 참여했습니다. **ArgoCD GitOps**로 Git 저장소의 manifest 변경이 곧 클러스터 상태에 반영되고, GitHub Actions의 이미지 빌드 → 매니페스트 자동 업데이트 → ArgoCD 자동 동기화 흐름으로 배포가 **PR 머지 → 운영 반영**까지 사람 개입 없이 완료되는 구조입니다.

성과

- 초거대 SaaS 정부과제 심사 통과
- 12개 마이크로서비스 기반 차세대 백엔드 아키텍처 설계 참여, 담당 도메인 설계·구현
- Docker Stack → Kubernetes 전환으로 배포 자동화 및 운영 효율화
- 서비스 단위 독립 배포 가능 구조 확보, 도메인별 릴리즈 사이클 분리

암호화 시스템 구축

- Springboot
- Java
- MyBatis
- Interceptor
- ISO27001

프로젝트 요약

- 개발 기간: 2025.06 ~ 2025.07
- 개발 내용:
 - MyBatis Interceptor 기반 자동 암호화 시스템 설계
 - DB 레벨 자동 암호화로 서비스 로직 무수정 적용
 - 비즈니스 코드와 보안 로직의 관심사 분리
- 핵심 기술: MyBatis Plugin
- 역할: 설계, 개발 전담

프로젝트 배경

개인정보보호법 및 ISO 인증 요건에 따라 DB 내 민감 데이터(이름·연락처·주소·식별번호 등) 암호화가 필요했습니다. 그러나 **수년간 누적된 기존 서비스 전반의 로직을 수정**하는 방식은 영향 범위가 광범위했고, 모든 도메인 코드의 회귀 테스트가 필요하다는 리스크가 컸습니다.

프로젝트 설명

서비스 코드를 건드리지 않고 어떻게 암호화를 적용할 것인가?

도메인 코드에 `EncryptionService.encrypt(...)` 를 명시적으로 호출하는 방식은 영향 범위가 크고, 신규 컬럼이 추가될 때마다 일일이 적용해야 하는 누락 리스크가 있었습니다.

이에 **MyBatis Interceptor**를 활용한 **DB 레벨 자동 암호화**를 설계했습니다:

- INSERT/UPDATE 시점에 암호화 대상 컬럼을 자동 암호화
- SELECT 결과 매핑 시점에 자동 복호화
- 서비스 로직은 평문으로 다루는 것처럼 동작

이 구조의 장점은 **서비스 로직 수정 0줄**로 전체 암호화 적용이 가능하다는 점이었고, 신규 컬럼 추가 시에도 어노테이션 한 줄로 자동 적용되어 누락 리스크가 사라졌습니다.

성과

- 서비스 코드 무수정으로 전체 암호화 적용 완료
- ISO 27001 / 27017 / 27018 / 27701 4종 인증 획득에 기여 (암호화 통제 항목 한정)
- 신규 민감 컬럼 추가 시 어노테이션 한 줄로 자동 암호화 적용

Keystone 검색 서버 안정화

Elasticsearch

Migration

Performance

JVM

프로젝트 요약

- 개발 기간: 2025.02 ~ 2025.04
- 개발 내용:
 - ES 2.x → 8.x 마이그레이션 (신규 클러스터 구축 후 데이터 이관)
 - wildcard 연산 제거 → ID 기반 term 검색 전환
 - N번 개별 조회 → 1회 배치 조회 전환
 - 불필요한 nested query 제거로 join 비용 절감
- 핵심 기술: Elasticsearch 2.x / 8.x, ES Slow Log 분석, JVM 힙 메모리 분석
- 역할: 원인 분석, 마이그레이션 설계 및 실행, 애플리케이션 쿼리 개선

프로젝트 배경

수억 건의 특허 문헌 전문(특히 상세설명 필드 최대 **200MB 이상**)을 저장한 ES 2.x 클러스터에서 wildcard 쿼리가 **CPU 100%**를 점유하며 전체 쿼리를 블로킹하는 현상이 발생했고, 이로 인해 **하루 3~4회 검색 서버 재기동**이 필요했습니다. 검색이 SaaS의 핵심 기능이기때 매일 발생하는 다운타임은 직접적인 사용자 이탈로 이어지는 상황이었습니다.

프로젝트 설명

근본 원인을 어떻게 진단했는가?

"ES가 느리다"는 증상만으로는 해결할 수 없어 ES Slow Log 분석부터 시작했습니다. 분석 결과 두 가지 복합 원인을 파악했습니다:

1. **wildcard 쿼리의 term enumeration**이 대용량 text 필드에서 CPU를 과점유
2. **ES 2.x의 field data 구조**가 힙 메모리를 압박하여 GC가 빈번히 발생

ES 2.x → 8.x 마이그레이션을 왜 선택했는가?

쿼리 튜닝만으로도 일부 개선은 가능했지만, ES 2.x는 이미 **EOL** 상태로 보안 패치도 받을 수 없었고, doc_values 기반의 field data 처리 방식 등 8.x의 구조적 개선을 따라가지 못했습니다. 운영 안정성과 장기 유지보수성을 고려하면 마이그레이션이 정답이었습니다.

다만 수억 건의 인덱스를 **무중단으로 이관**해야 했기 때문에, 신규 8.x 클러스터를 별도로 구축한 뒤 데이터 이관 → 듀얼 라이트 → 트래픽 스위치 순서로 마이그레이션을 진행했습니다.

쿼리 자체를 어떻게 개선했는가?

마이그레이션과 별개로, 애플리케이션 레벨의 쿼리 패턴도 함께 개선했습니다:

- **wildcard 연산 제거 → ID 기반 term 검색**: 후행 와일드카드 검색이 대부분 ID prefix였기에, ID 필드를 keyword 타입으로 분리해 term 검색으로 전환
- **N번 개별 조회 → 1회 배치 조회**: Family 확장 기능에서 자식 문헌을 1건씩 조회하던 로직을 mget/terms 쿼리 단일 호출로 변경
- **불필요한 nested query 제거**: 평탄화 가능한 구조는 평탄화하여 nested join 비용 절감

성과

- 검색 서버 재기동 빈도 **3~4회/일 → 0회**
- Family 확장 기능 속도 약 **81% 단축** (1분 18초 → 15초)
- ES 8.x 마이그레이션으로 **EOL 보안 리스크 해소** 및 장기 유지보수 가능 구조 확보

비로그인 문헌 공유 시스템 구축

Springboot

Java

UUID

JWT

BatchJob

프로젝트 요약

- 개발 기간: 2024.06 ~ 2024.12
- 개발 내용:
 - 비회원도 접근 가능한 문헌 공유 링크 시스템 설계
 - 유료 구독 상태 연동 접근 제어
 - 일 배치 기반 만료 링크 정리
 - 전체 접근 로그 기록
- 핵심 기술: UUID, JWT 비교 검토, Spring Batch
- 역할: 설계 및 개발 주도

프로젝트 배경

기존 서비스는 **로그인 사용자만 문헌 열람이 가능**한 구조여서, 사용자가 검색 결과를 동료에게 공유하려 해도 받는 사람이 회원 가입을 해야만 볼 수 있었습니다. 이로 인해 비회원 대상 서비스 확산과 신규 고객 유입 경로에 명확한 한계가 있었습니다.

공유 링크 자체는 단순한 기능처럼 보이지만, **유료 구독 서비스**라는 특성상 "구독이 해지된 후에도 링크가 유효하면 어떻게 되는가", "링크가 영구적으로 외부에 노출되어도 되는가" 등 결정해야 할 정책이 많았습니다.

프로젝트 설명

JWT인가 UUID인가?

처음에는 JWT가 자연스러운 선택지처럼 보였습니다. 만료 시간을 페이로드에 담을 수 있고, 서명 검증으로 위조를 막을 수 있기 때문입니다.

그러나 두 가지 결정적인 단점이 있었습니다:

1. **링크 영구 유지 요건과 충돌**: 비즈니스 요건상 "한 번 공유한 링크는 사용자가 폐기하기 전까지 유지"되어야 하는데, JWT 만료 시간 갱신이 어려움
2. **URL 길이 제한**: JWT는 페이로드가 커서 URL이 길어지고, 일부 메신저/이메일에서 잘리는 이슈

이에 **UUID 기반 링크 토큰**을 채택했습니다. 토큰은 짧고, 만료 정책은 DB에서 관리하므로 비즈니스 요건 변경에 유연합니다. 위조 방지는 UUID v4의 122bit 랜덤 엔트로피(사실상 추측 불가능한 키 공간)로 대응하고, 모든 접근을 로그로 기록해 사후 추적이 가능하도록 했습니다.

구독 해지 시 공유 링크를 어떻게 처리할 것인가?

유료 구독 사용자가 만든 공유 링크를 구독 해지 후에도 계속 유효하게 두면, 사용자가 "구독 해지 후에도 무료로 콘텐츠를 무한 배포"할 수 있는 우회 경로가 됩니다.

이에 **구독 상태와 공유 링크의 라이프사이클을 연동**하여, 구독 해지 시 해당 사용자의 모든 공유 링크를 자동 무효화하도록 설계했습니다. 또한 모든 접근에 대해 로그를 기록하여 사후 분석이 가능하도록 했습니다.

만료 링크 정리는 어떻게 할 것인가?

만료된 링크를 DB에 영구 보관하면 테이블 비대화로 검색 성능이 저하됩니다. 반면 실시간 삭제는 부하가 분산되지 않습니다.

일 배치(Spring Batch)로 만료 링크 정리를 일괄 처리하도록 설계했습니다. 트래픽이 적은 시간대에 한 번 처리되므로 운영 부하가 분산됩니다.

성과

- 오픈 1개월 만에 50만 건 공유 링크 생성
- 신규 고객 유입 경로 확보로 해당 분기 조직 OKR 'S' 등급 달성에 기여

- 구독 상태 연동 무효화 정책으로 콘텐츠 보안 유지

대용량 문헌 상세보기 성능 개선

Springboot

Java

DataCompression

프로젝트 요약

- 개발 기간: 2023.04 ~ 2023.06
- 개발 내용:
 - 1.5MB 이상 대용량 특허 문헌 데이터 응답 크기 축소
 - 비즈니스 로직 레벨 데이터 가공 및 압축 처리
 - 가공 데이터 사전 적재로 조회 시점 가공 비용 제거
- 핵심 기술: 데이터 압축, 응답 페이로드 최적화
- 역할: 원인 분석, 설계 및 개발

프로젝트 배경

1.5MB 이상의 대용량 특허 문헌 데이터를 사용자가 상세보기로 요청할 때 **Timeout 에러가 빈번하게 발생**하여 사용자 이탈이 증가하고 있었습니다. 특허 검색 서비스에서 상세보기는 핵심 사용자 동선의 종착점이기때, 이 단계에서의 실패는 곧 검색 세션 전체의 실패였습니다.

프로젝트 설명

응답 크기 자체를 어떻게 줄일 것인가?

Timeout의 1차 원인은 단순히 응답 페이로드가 너무 크다는 점이었습니다. 비즈니스 로직 레벨에서 데이터 가공 및 압축 처리를 적용하여 응답 크기를 축소했습니다. 클라이언트가 실제로 필요로 하는 필드만 추리고, 텍스트 압축을 적용하여 평균 95%의 크기 감소를 달성했습니다.

원본 DB의 조회 비용은 어떻게 줄일 것인가?

응답 크기를 줄여도 매 요청마다 원본 DB에서 1.5MB를 읽어와 가공하는 비용이 크다면 Timeout 위험은 여전히 남습니다.

이에 **가공된 데이터를 조회 전용 저장소에 사전 적재**하여, 상세보기 요청 시 가공 단계 없이 바로 응답할 수 있도록 했습니다.

성과

- 응답 데이터 크기 **평균 95% 축소**
- Timeout 문제 해결, 응답 속도 **75% 단축**
- 상세보기 단계에서의 사용자 이탈 감소

기타 주요 활동

어드민 기능 비동기 분리 (2022.09 ~ 2023.04)

RabbitMQ

AsyncDecoupling

- RabbitMQ를 도입하여 어드민 기능을 핵심 서비스로부터 비동기 분리
- 어드민 장애가 핵심 서비스로 전파되던 구조 해소
- **장애 전파 방지** 및 서비스 안정성 향상

회원 서비스 후처리 로직 리팩토링 (2023.03 ~ 2023.04)

CommandPattern Refactoring

- 회원 가입·탈퇴·정보 변경 후 발생하는 다수의 후처리 로직이 서비스 코드에 산재
- **Command Pattern**을 적용하여 후처리 로직을 명령 객체로 통합
- 신규 후처리 추가 시 명령 객체 1개 추가만으로 적용 가능한 구조 확보

AI 도구 팀 도입 (2025)

ClaudeCode DeveloperProductivity

- 백엔드 팀 10명에 **Claude Code** 도입을 주도
- 하네스 엔지니어링(PR 자동 작성 등)을 통해 팀 전체의 코드 품질 및 개발 생산성 향상
- AI 도구를 단순 보조가 아닌 **개발 워크플로우의 일부**로 정착

개발 문화 개선 (2022.08 ~)

DocumentationCulture

- API 명세서, Infra 구성도 등 핵심 기술 문서의 표준화 및 작성 주도
- 신규 입사자 온보딩 시간 단축 및 팀 내 지식 공유 활성화

기술 스택

카테고리	기술
Language	Java, Kotlin
Framework	Spring Boot, Spring MVC, MyBatis, JPA, Spring Batch
Database	MySQL, Redis
Search Engine	Elasticsearch
Message Queue	RabbitMQ, Amazon SQS
Infra & DevOps	AWS, Kubernetes, Docker, ArgoCD, GitHub Actions
Tools	Git, GitHub, Claude Code

학력

- **학점은행제** 컴퓨터공학과 졸업 (2021)
- **명지전문대학** 컴퓨터전자과 졸업 (2021)

자격증 / 어학

- 정보처리기사 (2021.11)
- JLPT N2 (2021.08)